

1 Introduction

La **complexité algorithmique temporelle asymptotique** dans le **pire des cas**, ou tout simplement la **complexité**, est concept qui permet de comparer les algorithmes afin de trouver celui qui est le plus efficace.

Le critère le plus important est le **temps d'exécution**. si deux algorithmes font la même chose, mais un est plus rapide que l'autre. On dit qu'il est plus **efficace**.

Puisque le temps d'exécution dépend sur le matériel utilisé (processeur, RAM, ...), on doit définir une méthode pour calculer la complexité qui est indépendante du matériel.

2 Complexité temporelle

C'est tout simplement le **temps** que va prendre un algo pour terminer son exécution.

Definition : On appelle complexité d'un algorithme le **nombre asymptotique d'opérations de base** qu'il doit effectuer en fonction de la taille de l'entrée, dans le pire des cas.

2.1 Opérations de base

Aussi appelé instructions élémentaires, sont les instructions les plus simple, chaque une à un coups de 1 :

- Les opérations arithmétiques (addition, soustraction, multiplication, division, modulo, partie entière, ...)
- La comparaisons de données (égalité, infériorité, ...)
- Le transferts de données (Affectation, lecture et écriture dans un emplacement mémoire)
- Les instructions de contrôle (branchement conditionnel et inconditionnel, appel à une fonction auxiliaire, ...)

2.2 Calcul de complexité

Il suffit de compter le nombre d'opérations de bases qui va exécuter un algorithme, dans le **pire des cas**.

Exemples :

```
1 def fct(a, b):
2     c = 5           # 1 (affectation)
3     d = a + b       # 2 (affectation et addition)
4     return d / c    # 1 (division)
```

La complexité est : 5

```
1 def abs(a):
2     if a < 0:        # 1 (comparaison)
3         a = -a      # 2 (affectation est multiplication)
4     return a         # 0 (return ne compte pas)
```

La complexité est : 3

3 Règles de calcul

3.1 Algorithme itérative

Dans un programme purement séquentiel, où il n'y a pas de récursivité, les temps d'exécution s'additionnent. Ainsi :

- Dans le cas d'une instruction alternative :

```
1 if bloc A:
2     bloc B
3 else:
4     bloc C
```

$$T(n) = T_A(n) + \text{Max}(T_B(n), T_C(n))$$

- dans le cas d'une instruction répétitive :

```
1 boucle bloc A:
2     bloc B
```

$$T(n) = (\text{nombre d'itérations})(T_A(n) + T_B(n))$$

Pour Calculer le nombre d'itérations d'une boucle *while*. Il faut identifier les invariants de boucle (les variable qui ne change pas dans la boucle) et les variant de boucle (les variable qui change pas dans la boucle d'une manière prédictible).

Puis utiliser ces variable pour déterminer *f* le nombre d'itérations

```
1 def fct2(n):
2     i, x = 1, 0    # 2
3     while i < n:   # 1
4         i = i * 2   # 2
5         x = x + 1   # 2
6     return x
```

$T(n) = 2 + 5f$ avec *f* le nombre d'itération.

trouver le nombre d'itérations :

soit i_0 et x_0 les valeur de *i* et *x* avant la boucle,

et i_f et x_f les valeur de *i* et *x* après la boucle.

On a $i_0 = 1, x_0 = 0, i_{f-1} < n$ et $n \leq i_f$

On a aussi $i_{k+1} = i_k * 2$ Donc $i_f = i_0 \times 2^f = 2^f$

Donc $2^{f-1} < n$ et $n \leq 2^f \Rightarrow f-1 < \log(n) \leq f \Rightarrow f = O(\log(n))$

Conclusion : $T(n) = 2 + 5f = O(\log(n))$

3.2 Algorithme récursive

Pour les fonctions récursives, le calcul de la complexité revient toujours à calculer la valeur d'une suite.

```
1 def fact(n):
2     if n==0: return 1    # 1
3     return n * fact(n-1) # 1 + T(n-1)
```

Soit $T(n)$ la complexité de *fact(n)*.

On a $T(n) = 1 + 1 + T(n-1) = T(n-1) + O(1)$

Donc $T(n) = T(n-2) + 2 \times O(1) = \dots = T(0) + n \times O(1) = O(n)$

4 Ordre de grandeur

Les machines modernes sont très rapides, La différence entre le temps d'exécution des algorithmes avec des complexités petites est négligeable.

Donc On va seulement se concentrer sur des nombres très grands, proche de l'infini, d'où vient l'étude asymptotique de la complexité.

Comme exemple la différence entre n et $n + 1000$ est négligeable si n est très grande.

4.1 Grand-O

Grand-O, aussi appelé **dominance**, est un outil pour faire une comparaison asymptotique de deux fonctions.

Définition : Étant donné deux fonctions $f(n)$ et $g(n)$, on dit que $f(n) = O(g(n))$ lorsqu'il existe des constantes $N > 0$ et $C > 0$ telles que :

$$\forall n < N \quad |f(n)| \leq C|g(n)|$$

4.2 Taux de croissance

La phrase " $f(n)$ est $O(g(n))$ " ($f(n) = O(g(n))$) signifie donc que le taux de croissance de $f(n)$ est plus petit ou égal au taux de croissance de $g(n)$.

On peut utiliser la notation O pour ordonner les fonctions à partir de leur taux de croissance :

$$O(1) < O(\log(n)) < O(n) < O(n \log(n)) < O(n^2) < O(2^n) < O(n^n)$$

4.3 Les complexités algorithmique

Tout les réponses des questions sur la complexité doivent être avec un Grand-O.

Voici un tableau qui liste (presque) toutes les complexités algorithmiques qu'on peut trouver, triées selon leur ordre de croissance.

$O(\log n)$	logarithmique
$O(n)$	linéaire
$O(n \log n)$	quasi-linéaire
$O(n^2)$	quadratique
$O(n^k) \quad (k \geq 2)$	polynomiale
$O(k^n) \quad (k > 1)$	exponentielle

4.4 Règles d'utilisation

- Si $f(n)$ est un polynôme de degré d , alors $f(n) = O(n^d)$
- On utilise toujours la plus petite classe de complexité valable.
par exemple : On dit $2n = O(n)$ et non $2n = O(n^2)$
- Si $f_1 = O(g_1)$ et $f_2 = O(g_2)$ alors $f_1 + f_2 = \max(O(g_1), O(g_2))$

5 Exemples

```
1 def fact(n):
2     if n < 0: # A
3         return 'erreur' # A
4     elif n == 0: # B
5         return 1 # B
6     else: # C
7         f = 1 # O(1)
8         for i in range(1, n+1): # D
9             f = f * i # D
10        return f # C
```

$$\begin{aligned} T_{fact(n)} &= \max(T_A(n), T_B(n), T_C(n)) \\ T_A(n) &= 1 = O(1) \\ T_B(n) &= 1 = O(1) \\ T_C(n) &= 1 + T_D(n) = 1 + n \times 2 = O(n) \\ \text{Donc } T_{fact(n)} &= \max(O(1), O(1), O(n)) = O(n) \end{aligned}$$

```
1 def som(n):
2     x = 0 # O(1)
3     for i in range(n): # A
4         for j in range(n): # B
5             x += 1 # C
6     return x
```

$$\begin{aligned} T_{som(n)} &= O(1) + T_A(n) \\ T_{som(n)} &= O(1) + n \times T_B(n) \\ T_{som(n)} &= O(1) + n \times n \times T_C(n) \\ T_{som(n)} &= O(1) + n \times n \times 2 = O(1) + 2n^2 = O(n^2) \end{aligned}$$

```
1 def recherche(L, x):
2     n = len(L) # O(1)
3     for e in L: # A
4         if e == x: # B
5             return True #
6     return False
```

On doit se rappeler qu'on veut trouver la complexité dans le pire des cas.

Ici, le pire des cas est "**x ne se trouve pas dans L**"

$$\begin{aligned} T_{recherche}(n) &= O(1) + T_A(n) \\ T_{recherche}(n) &= O(1) + n \times T_B(n) \\ T_{recherche}(n) &= O(1) + n \times O(1) = O(n) \end{aligned}$$

6 Théorème maître

Pour les fonctions récursives, parfois le calcul de la suite n'est pas évident. Dans ce cas on peut utiliser le théorème suivant :

- Pour $T(n) = aT(n/b) + O(n^d)$, le terme général est :

$$T(n) = \begin{cases} O(n^d) & \text{si } d > \log_b a \\ O(n^d \log n) & \text{si } d = \log_b a \\ O(n^{\log_b a}) & \text{si } d < \log_b a \end{cases}$$